

# Fase 1: comprende

## Checkpoint Metacognitivo — COMPRENDE

**¿Puedes explicar en tus propias palabras por qué usar la cabeza de la lista enlazada como tope hace que push y pop sean  $O(1)$ ?**

por que no mueves nada del arreglo, sino que simplemente lo insertas o mueves sin tener que recorrer o dividir algo

**Si alguien te pregunta "¿para qué sirve una pila y dame dos ejemplos reales?", ¿puedes responder sin consultar estas notas? Si no, relee la sección de aplicaciones.**

eso creo, ya que una pila a final de cuentas es una estructura de datos que como cualquier estructura de datos organiza elementos y los dos ejemplos reales que se me ocurren que podría decir sería usar el ctrl z y ctrl y en alguna página tal vez y también la implementación del historial de búsqueda en navegadores

**¿Cuál es la diferencia funcional entre peek() y pop()? ¿En qué situación usarías uno vs el otro en el verificador de paréntesis?**

la diferencia es que peek devuelve sin eliminar un valor mientras que pop devuelve y elimina, aunque aun no visualizo en que situación podría usar cada uno

**Alerta IA: Si le pediste a la IA que "te explicara las pilas", ¿puedes reproducir esa explicación con tus propias palabras? Si no, vuelve a preguntar pidiendo que use analogías más simples.**

una pila (stack) es una estructura de datos que funciona con una regla muy simple: "El último en entrar es el primero en salir."  
A esto se le llama LIFO (Last In, First Out).

Analogía sencilla: una pila de platos  
Imagina que tienes una pila de platos en la cocina:  
Pones un plato encima.  
Luego otro encima.  
Luego otro más.

## Fase 2: aplica

### 1.- Proyecto — Sistema de Edición CodeLeap con Pila propia:

Archivo de código con: (1) Clase Nodo y Clase Pila completamente implementadas con comentarios explicando cada línea clave, (2) Clase EditorCodeLeap con métodos escribir, deshacer, rehacer y mostrarHistorial, (3) Escenario de prueba del enunciado ejecutado con salida visible, (4) Comentario al inicio explicando con tus palabras por qué se necesitan dos pilas para el sistema deshacer/rehacer:

```
#include <iostream>
#include <string>
using namespace std;

//== ETAPA 1: Construir la Pila (TAD) ==

// parte 1 clase nodo
class nodo{
public:
    string dato; // lo que se guarda
    nodo * siguiente; // apunta al siguiente nodo

    //parte constructora (osea crea un nodo y su puntero inicialmente en null)

    nodo(const string& valor) : dato(valor), siguiente(nullptr) {}
};

//2. Clase Pila con atributo tope y contador inicializados.
class pila{
private:
    nodo * tope; //nodo cola por asi decir
    int cont;

public:
    //parte constructora de la pila
    pila() : tope(nullptr), cont(0) {}
```

```

//3. Método push(valor)

void push(const string& valor) {
    nodo * nuevo = new nodo(valor); //generacion de nuevo nodo
    nuevo -> siguiente = tope; //engancha al nodo que habia
    tope = nuevo; //el nodo nuevo esta arriba
    cont++; //ahora hay un nodo mas
}

//4. Método pop()

string pop() {
    if (estavacia()){ //devolvemos string vacio si es que la lista esta
vacía
        return "";
    }
    nodo * temp = tope; // nodo que se retira
    string valor = temp->dato; //se rescata el dato
    tope = tope->siguiente; //el de abajo sube a ser tope
    delete temp; //se libera memoria
    cont--; // se resta un nodo
    return valor; // se regresa lo que retiramos
}

//5. Método peek()

string peek() const {
    if (estavacia()) return "";
    return tope->dato;
}

//6. método estavacia()

bool estavacia() const {
    return tope == nullptr;
}

//7. método tamaño

int tamaño() const{
    return cont;
}

//parte destructora
~pila(){

```

```

        while (!estavacia()) {
            pop();
        }
    }
};

//ETAPA 2: EDITOR con DESHACER/REHACER

class editorcodeleap{
private:
    pila piladeshacer;
    pila pilarehacer;
    string estadoactual;

    //esta parte es para borrar rehacer ya que cuando escribes algo nuevo
    no tiene mucho sentido rehacer lo anterior
    void vaciarpila(pila& p) {
        while (!p.estavacia()) {
            p.pop();
        }
    }

public:
    editorcodeleap() : estadoactual("") {}

    //guardamos el estadoactual en deshacer antes de cambiarlo
    void escribir(const string& cambio) {
        //1. guarda el estado actual para luego poder devolverlo con ctrl+z
        if (!estadoactual.empty()) {
            piladeshacer.push(estadoactual);
        }
        //2. esta parte modifica el estado actual
        estadoactual += cambio + "\n";

        //3. esta parte crea una nueva "linea de tiempo", lo que significa que
        borra rehacer
        vaciarpila(pilarehacer);
    }

    void deshacer() {

```

```

    if (piladeshacer.estavacia()){
        return; // si no hay nada que deshacer no se deshace nada
    }

    // esta parte guarda el estado actual en rehacer (si es que luego
haces ctrl+y)
    if (!estadoactual.empty()){
        pilarehacer.push(estadoactual);
    }
    //esta parte hace que se vuelva a la version anterior
    estadoactual = piladeshacer.pop();
}

//parte de rehacer

void rehacer(){
    if (pilarehacer.estavacia()){
        return;
    }

    // esta parte guarda el estado actual para poder "regresar" si es
que luego deshaces
    if(!estadoactual.empty()){
        piladeshacer.push(estadoactual);
    }
    //esta parte recupera la version de lo antes deshecho
    estadoactual = pilarehacer.pop();
}

// parte mostrarhistorial(). esta parte imprime el texto actual y los
tamaños de las pilas

void mostrarHistorial() const {
    cout << "=====\n";
    cout << "ESTADO ACTUAL:\n";
    if (estadoactual.empty()) cout << "(vacio)\n";
    else cout << estadoactual;

    cout << "-----\n";
    cout << "Pila Deshacer (tam): " << piladeshacer.tamano() <<
"\n";

```

```

        cout << "Pila Rehacer (tam): " << pilarehacer.tamano() <<
"\n";
        cout << "=====\n\n";
    }

    string obtenerEstadoActual() const {
        return estadoactual;
    }
};

int main() { //aquí se ejecuta el escenario de prueba completo
    editorcodeleap editor;

    cout << "== 1) Escribir 3 lineas ==\n";
    editor.escribir("def funcion():");
    editor.escribir("    x = 5");
    editor.escribir("    return x");
    editor.mostrarHistorial();

    cout << "== 2) Deshacer dos veces ==\n";
    editor.deshacer(); // quita "return x"
    editor.deshacer(); // quita "x = 5"
    editor.mostrarHistorial();

    cout << "== 3) Rehacer una vez ==\n";
    editor.rehacer(); // recupera "x = 5"
    editor.mostrarHistorial();

    return 0;
}

```

2: Verificador de Paréntesis para el Linter de CodeLeap  
 Método verificarBalanceo() integrado en la clase EditorCodeLeap, con las 5 pruebas del enunciado ejecutadas y sus resultados. Incluye un comentario personal de 3-4 líneas explicando por qué una pila es esencial aquí y un simple contador no funcionaría.

```
#include <iostream>
```

```

#include <string>
using namespace std;

//== ETAPA 1: Construir la Pila ==

// parte 1 clase nodo
class nodo{
    public:
        string dato; // lo que se guarda
        nodo * siguiente; // apunta al siguiente nodo

        //parte constructora (osea crea un nodo y su puntero inicialmente
        en null)

        nodo(const string& valor) : dato(valor), siguiente(nullptr) {}
};

//2. Clase Pila con atributo tope y contador inicializados.
class pila{
private:
    nodo * tope; //nodo cola por asi decir
    int cont;

public:
    //parte constructora de la pila
    pila() : tope(nullptr), cont(0) {}

    //3. Método push(valor)

    void push(const string& valor) {
        nodo * nuevo = new nodo(valor); //generacion de nuevo nodo
        nuevo->siguiente = tope; //engancha al nodo que habia
        tope = nuevo; //el nodo nuevo esta arriba
        cont++; //ahora hay un nodo mas
    }

    //4. Método pop()

    string pop() {
        if (estavacia()){ //devolvemos string vacio si es que la lista esta
            vacia

```

```

        return "";
    }

    nodo * temp = tope; // nodo que se retira
    string valor = temp->dato; //se rescata el dato
    tope = tope->siguiente; //el de abajo sube a ser tope
    delete temp; //se libera memoria
    cont--; // se resta un nodo
    return valor; // se regresa lo que retiramos
}

```

//5. Método peek()

```

string peek() const {
    if (estavacia()) return "";
    return tope->dato;
}

```

//6. metodo estavacia()

```

bool estavacia() const {
    return tope == nullptr;
}

```

//7. metodo tamaño

```

int tamanio() const{
    return cont;
}

```

//parte destructora

```

~pila(){
    while (!estavacia()){
        pop();
    }
}

};

```

//ETAPA 2: EDITOR con DESHACER/REHACER

```

class editorcodeleap{
private:

    pila piladeshacer;

```



```

pila pilarehacer;
string estadoactual;

//esta parte es para borrar rehacer ya que cuando escribes algo nuevo
no tiene mucho sentido rehacer lo anterior
void vaciarpila(pila& p){
    while (!p.estavacia()){
        p.pop();
    }
}

public:editorcodeleap() : estadoactual(""){}

//guardamos el estadoactual en deshacer antes de cambiarlo
void escribir(const string&cambio) {
    //1. guarda el estado actual para luego poder devolverlo con ctrl+z
    if (!estadoactual.empty()){
        piladeshacer.push(estadoactual);
    }
    //2. esta parte modifica el estado actual
    estadoactual +=cambio + "\n";

    //3.esta parte crea una nueva "linea de tiempo", lo que significa que
    borra rehacer
    vaciarpila(pilarehacer);
}

void deshacer() {
    if (piladeshacer.estavacia()){
        return; // si no hay nada que deshacer no se deshace nada
    }

    // esta parte guarda el estado actual en rehacer (si es que luego
    haces ctrl+y)
    if (!estadoactual.empty()){
        pilarehacer.push(estadoactual);
    }
    //esta parte hace que se vuelva a la version anterior
    estadoactual = piladeshacer.pop();
}

```

```

//parte de rehacer

void rehacer(){
    if (pilarehacer.estavacia()){
        return;
    }

    // esta parte guarda el estado actual para poder "regresar" si es
que luego deshaces
    if(!estadoactual.empty()){
        piladeshacer.push(estadoactual);
    }
    //esta parte recupera la version de lo antes deshecho
    estadoactual = pilarehacer.pop();
}

// parte mostrarhistorial(). esta parte imprime el texto actual y los
tamaños de las pilas

void mostrarHistorial() const {
    cout << "=====\n";
    cout << "ESTADO ACTUAL:\n";
    if (estadoactual.empty()) cout << "(vacio)\n";
    else cout << estadoactual;

    cout << "-----\n";
    cout << "Pila Deshacer (tam): " << piladeshacer.tamano() <<
"\n";
    cout << "Pila Rehacer (tam): " << pilarehacer.tamano() <<
"\n";
    cout << "=====\n\n";
}

string obtenerEstadoActual() const {
    return estadoactual;
}

// =====
// NUEVO: verificador de brackets
// =====
pair<bool, string> verificarBalanceo(const string& codigoFuente) {
    pila p; // pila LOCAL solo para brackets

```

```

auto esApertura = [](char c){
    return c == '(' || c == '[' || c == '{';
};

auto esCierre = [](char c){
    return c == ')' || c == ']' || c == '}';
};

auto aperturaEsperada = [](char cierre) -> char {
    if (cierre == ')') return '(';
    if (cierre == ']') return '[';
    return '{'; // para '}'
};

auto cierreEsperado = [](char apertura) -> char {
    if (apertura == '(') return ')';
    if (apertura == '[') return ']';
    return '}'; // para '{'
};

for (int i = 0; i < (int)codigoFuente.size(); i++){
    char c = codigoFuente[i];

    if (esApertura(c)){
        p.push(string(1, c));
    }
    else if (esCierre(c)){
        if (p.estavacia()){
            return {false, "INVALIDO: cierre sin apertura: '" +
string(1, c) + "' en posicion " + to_string(i)};
        }

        string top = p.pop(); // top deberia ser "(" o "["
o "{"

        char apertura = top.empty() ? '\0' : top[0];

        char esperado = aperturaEsperada(c);
        if (apertura != esperado){
            return {false,
                    "INVALIDO: bracket incorrecto al cerrar. Se
esperaba '" +
                    string(1, cierreEsperado(apertura)) + "' pero
se encontro '" +

```

```

        string(1, c) + " en posicion " +
to_string(i));
    }
}

    if (!p.estavacia()){
        return {false, "INVALIDO: faltan cierres. Quedaron
aperturas sin cerrar (ej: '" + p.peek() + "')"};
    }

    return {true, "VALIDO: brackets balanceados correctamente."};
}
};

int main() { //aquí se ejecuta el escenario de prueba completo
    editorcodeleap editor;

    cout << "== 1) Escribir 3 lineas ==\n";
    editor.escribir("def funcion():");
    editor.escribir("    x = 5");
    editor.escribir("    return x");
    editor.mostrarHistorial();

    cout << "== 2) Deshacer dos veces ==\n";
    editor.deshacer(); // quita "return x"
    editor.deshacer(); // quita "x = 5"
    editor.mostrarHistorial();

    cout << "== 3) Rehacer una vez ==\n";
    editor.rehacer(); // recupera "x = 5"
    editor.mostrarHistorial();

    // =====
    // PRUEBAS de balanceo
    // =====
    cout << "\n== Pruebas balanceo ==\n";
    string casos[] = { "({}[[])", "({[[]})", "({[[]})", "(((" , ")"))" };

    for (auto &s : casos){
        auto r = editor.verificarBalanceo(s);
        cout << s << " -> " << (r.first ? "VALIDO" : "INVALIDO") << " |
" << r.second << "\n";
    }
}

```

```
}  
  
return 0;  
  
//aqui es mas util usar pilas ya que el contador solo cuenta los  
valores pero una pila puede diferenciar valores por asi decir  
}
```

## Checkpoint Metacognitivo — APLICACIÓN

**¿Puedes explicar sin mirar el código por qué se necesitan DOS pilas para implementar Deshacer + Rehacer, y no una sola?**

por que una pila solo te deja ir en alguna direccion; la pila de deshacer guarda elementos para que puedas regresar y la de rehacer para que puedas avanzar

**¿Puedes trazar a mano el estado de la pila con la cadena "{}[]" para demostrar que el verificador la rechaza correctamente?**

aun no, me hace falta practicar o estudiar

## FASE 3: REFLEXIONA

### 3: Trade-offs y Decisiones Técnicas

Documento escrito (no código) de media página con: (1) Tu posición final sobre lista enlazada vs arreglo para pilas, justificada, (2) La decisión que tomarías para el historial del editor y por qué, (3) Un criterio personal para elegir cuándo usar pila vs otra estructura.

1.- las listas enlazadas y los arreglos son dos tipos de estructuras que suelen ser comunes a la hora de trabajar usando pilas, y una cosa que se nos ha enseñado durante en el curso es que usualmente no hay un trade-off mejor o peor que otro, sino que cada uno tiene sus ventajas o desventajas y que por ende, algunos son mas beneficiosos en ciertas situaciones que otros.

esto mismo se aplica a la hora de decidir con que tipo de estructura quieres trabajar usando pilas (arreglo o lista enlazada) y por eso mismo con lo visto en el prompt para realizar este apartado llegue a una conclusion.

los arreglos son mejores cuando necesitas:

- rendimiento constante alto
- si es que la pila crece mucho y ocupas espacio extra
- simplicidad y te quieres ahorrar problemas respecto a los punteros

y las listas enlazadas son mejores cuando necesitas:

- las funciones de push/pop
- implementar algo con bloques dispersos
- trabajar en un contexto donde no ocupes realocaciones

asi que por lo mismo, considero que no hay una mejor o peor sino que depende en que situacion te encuentres.

2.- usaria el texto completo ya que no estoy trabajando con archivos tan grandes

3.-a mi criterio es mejor usar arreglos cuando trabajas con elementos grandes (trabajar con arreglo dinamico) y es mejor usar listas enlazadas cuando ocupas usar bloques dispersos por alguna razon

## Checkpoint Metacognitivo — REFLEXIONA

**Explica en una oración: ¿por qué la pila de llamadas del sistema operativo usa exactamente el principio LIFO?**

por que la ultima funcion que se ejecuto debe de ser la primera en salir

**¿Podrías defender ante un colega tu elección de implementación (lista enlazada vs arreglo) para la pila de CodeLeap? ¿Qué criterio usaste?**

defenderme muy bien no estoy seguro pero pues a final de cuenta no estamos trabajando con varios archivos y menos con archivos grandes y pues necesitamos las funciones de principales de las pilas y pues los criterios que use fueron los que me dieron las actividades los cuales fueron usar listas enlazadas

## FASE 4: VALIDA

✓ **¿Push y Pop son correctos en secuencia?** Haz `push(1)`, `push(2)`, `push(3)` y luego tres `pop()`. ¿Obtienes 3, 2, 1 en ese orden?

→ Si no: revisa el orden en `push()` — ¿asignas siguiente ANTES de actualizar `tope`?

☐ **¿Pop() en pila vacía lanza error?** Crea una pila vacía y llama `pop()`. ¿Tu código lanza una excepción o error claro, no un null silencioso?

→ Si no: falta el guard de `estaVacía()` al inicio de `pop()` y `peek()`

✓ **¿Peek() no destruye la pila?** Después de `peek()`, ¿el tamaño de la pila sigue siendo el mismo?

→ Si no: estás moviendo `tope` en `peek()` cuando solo debes leer `tope.dato`

✓ **¿El sistema Deshacer/Rehacer mantiene consistencia?** Escribe 3 acciones, deshaz 2, rehaz 1, escribe 1 nueva. ¿La pila Rehacer se limpió al escribir la nueva acción?

→ Si no: falta vaciar `pilaRehacer` al escribir una nueva acción

✓ **¿El verificador rechaza `{{[()]}`?** Esta cadena tiene el mismo número de aperturas y cierres, pero está mal ordenada. ¿Tu verificador devuelve INVÁLIDO?

→ Si no: estás comparando contadores en lugar de comparar con el tope de la pila

✓ **¿El verificador detecta brackets sin cerrar?** Prueba con `"((("`. ¿Devuelve INVÁLIDO después de recorrer toda la cadena?

→ Si no: falta verificar que la pila esté vacía al TERMINAR de recorrer la cadena

## Checkpoint Metacognitivo — VALIDA

**¿Pasaron TODOS los casos de prueba del checklist? Si alguno falló, ¿ya identificaste la línea de código específica que lo causa?**

me faltó que en `pop()` mande error, o bueno es que no manda error como tal sino que manda un mensaje que yo puse que mandara

**¿Puedes explicar por qué el caso `"([)]"` es inválido sin ejecutar el código — solo razonando con el estado de la pila en cada paso?**

esto es por que cuando entra el ultimo bracket lo compara con el ultimo que quedo abierto